Cache Performance Part 2

Way Prediction

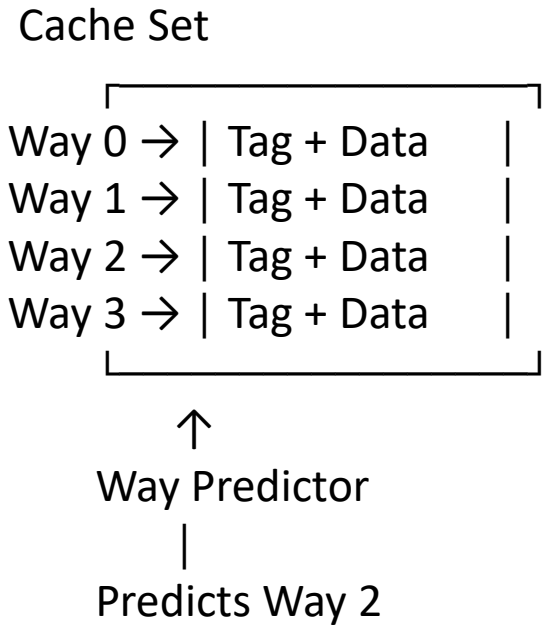
- Extra bits are kept in cache to predict the way, or block within the set of the next cache access
- Multiplexor is set early to select the desired block, and only a single tag comparison is performed that clock cycle
- A miss results in checking the other blocks for matches in subsequent clock cycles
- Alpha 21264 uses way prediction in its 2-way set-associative instruction cache. Simulation using SPEC95 suggested way prediction accuracy is in excess of 85%

Why is Way Prediction Needed?

- Consider a **4-way set-associative cache**. For every memory address:
- The address selects one cache **set**.
- The cache normally checks the **tags of all 4 ways** in parallel.
- One way is found to contain the requested data.
- The corresponding data is returned.
- Checking all four ways in parallel is fast, but it requires more **hardware, power, and energy**.
- **Way prediction** tries to predict the correct way before doing the full tag comparison.

Basic Idea

- Suppose a cache set has four ways:



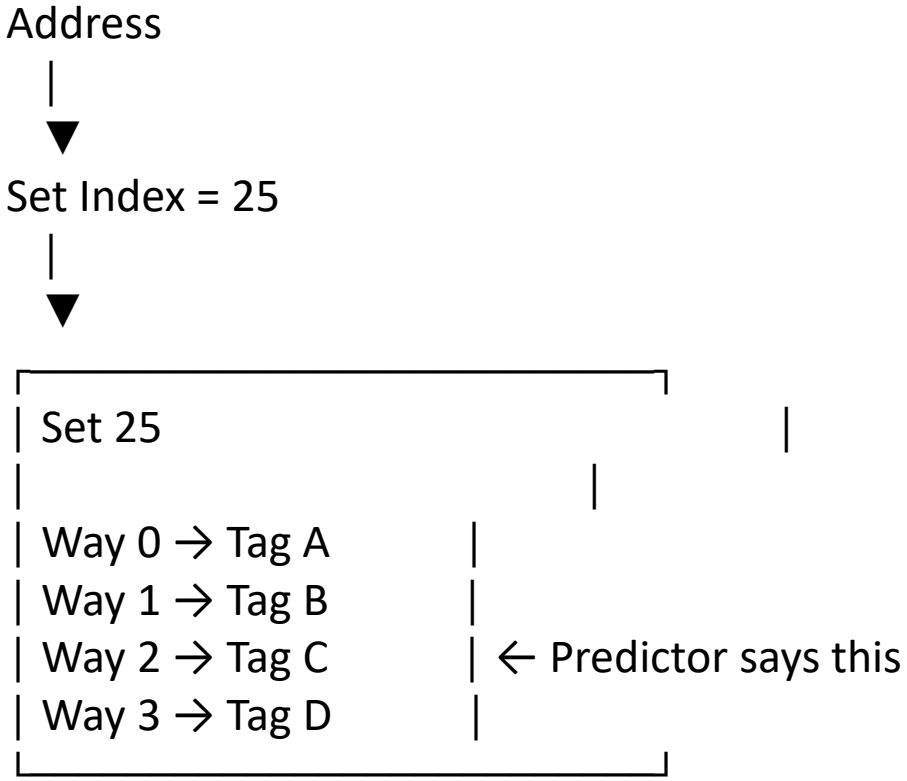
- If the predictor says **Way 2**, the processor first accesses Way 2.
- If Way 2 contains the required block → **cache hit with low access cost**.
- If not → the cache performs a normal lookup of the remaining ways.

Conventional Cache vs. Way-Predicted Cache

Feature	Conventional Set-Associative Cache	Way-Prediction Cache
Ways accessed initially	All ways	Predicted way
Tag comparisons	Multiple	Usually one initially
Access energy	Higher	Lower
Hit latency	Predictable	Can be lower
Hardware	Simpler	Requires predictor
Prediction failure	N/A	Additional lookup required
Best advantage	Simplicity	Power/energy reduction

Example

- Assume a **4-way cache**:



Suppose the requested address has **Tag C**.
The predictor predicts:

Set 25 → Way 2

The cache accesses Way 2 first:

Predicted Way = 2
Actual Way = 2
↓
HIT ✓

Only the predicted way needs to be activated initially.

What Happens When Prediction Is Wrong?

- Suppose the predictor predicts Way 2, but the requested block is actually in Way 1.

Prediction:
Set 25 → Way 2

Actual:
Set 25 → Way 1

The first access fails:

Way 2 → MISS

The cache then checks the other ways:

Way 0 → No
Way 1 → YES ✓
Way 3 → No

This is called a **way-prediction misprediction**.

How Does the Predictor Work?

- The predictor maintains information about which way was recently used for a particular set or address.
- A simple predictor could be:

Set Number	Predicted Way
0	Way 1
1	Way 3
2	Way 0
3	Way 2
4	Way 2
...	

- For example, if accesses to Set 4 frequently hit Way 2:

  Previous accesses:

  Set 4 → Way 2
  Set 4 → Way 2
  Set 4 → Way 2
  Set 4 → Way 2

the predictor learns:

Set 4 → predict Way 2

Common Way-Prediction Mechanisms

- Several approaches can be used:
- **1. Last-used way**
- Store the way that was accessed most recently.

Last access → Way 3
Next prediction → Way 3

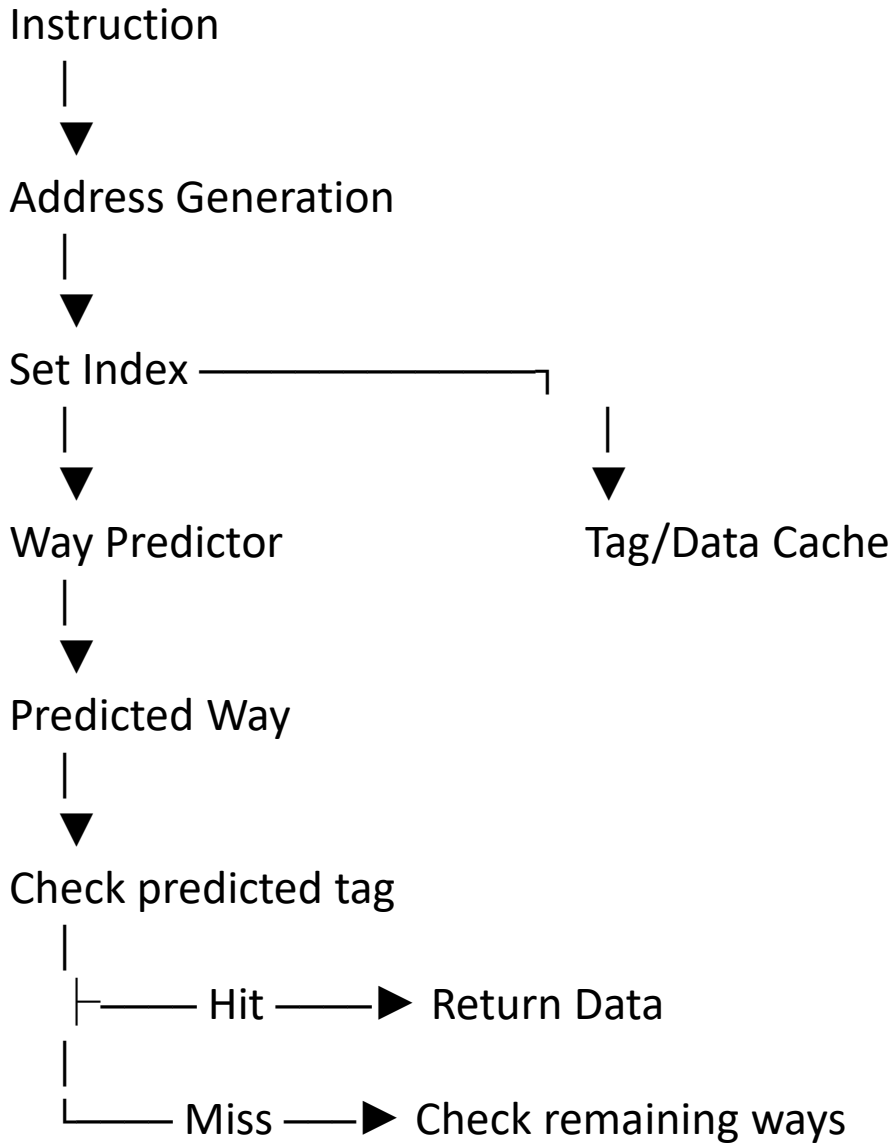
This works well when programs repeatedly access the same cache line.
2. MRU-based prediction
Use the **Most Recently Used (MRU)** way as the prediction.
3. History-based predictor
Maintain a small history table:

Address/Set → Recent Way Pattern
↓
2 → 1 → 2 → 1

The predictor attempts to learn the pattern.

Way Prediction and CPU Pipeline

- Way prediction is particularly useful in modern processors where cache latency is critical.
- A simplified pipeline could be:



Important Metrics

- When designing a way-prediction cache, three metrics are particularly important:
- **Prediction Accuracy**
- $Accuracy = \frac{Correct\ Predictions}{Total\ Predictions}$
- For example, if 950 out of 1000 predictions are correct:
- $Accuracy = 95\%$
- **Misprediction Rate**
- $Misprediction\ Rate = 1 - Accuracy$
- For 95% accuracy:
- $Misprediction\ Rate = 5\%$
- **Energy Reduction**
- Because fewer ways are activated during a typical access, dynamic energy can be significantly reduced, depending on cache organization and implementation.

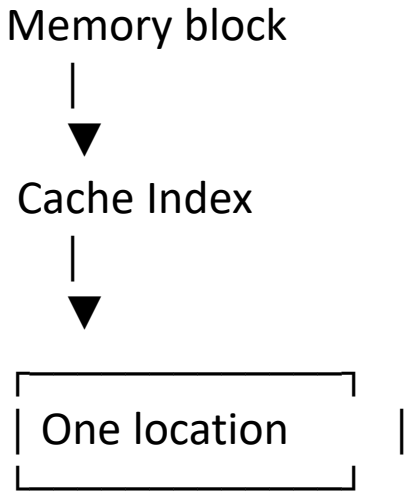
Pseudo-Associative Cache

Pseudo-Associative Cache

- A **pseudo-associative cache** is a cache organization that tries to get some of the benefits of a **set-associative cache** while keeping the **fast access time of a direct-mapped cache**.
- It is sometimes called a **column-associative cache**.

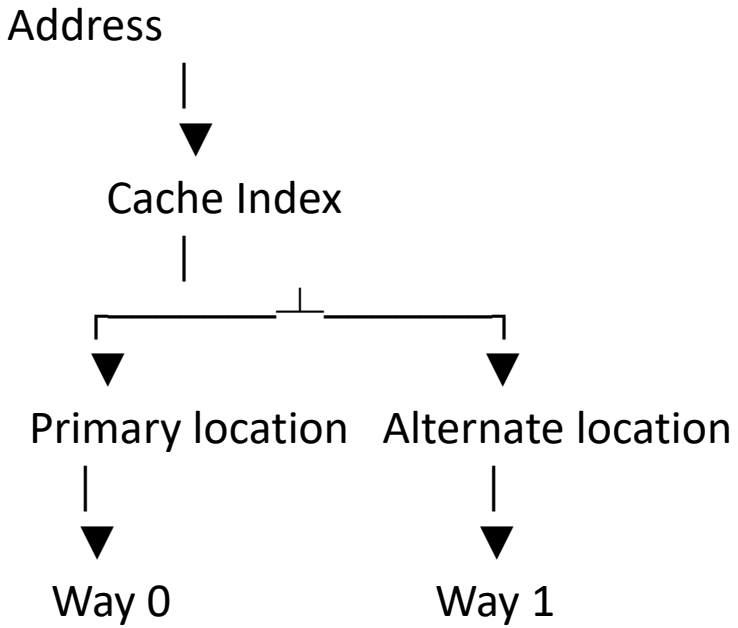
Basic idea

- A direct-mapped cache has only **one possible location** for each memory block:



This makes it fast, but two frequently used blocks can map to the same location and continuously evict each other.

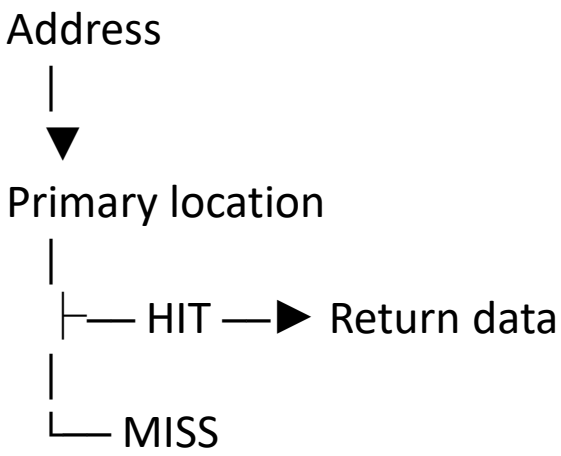
- A pseudo-associative cache adds a **second possible location**.



So, although it behaves somewhat like a 2-way associative cache, it does **not access both locations in parallel**.

How It Works

- Consider a cache with:
- Primary location
- Alternate location
- When a memory address arrives:
- **Step 1 — Check primary location**



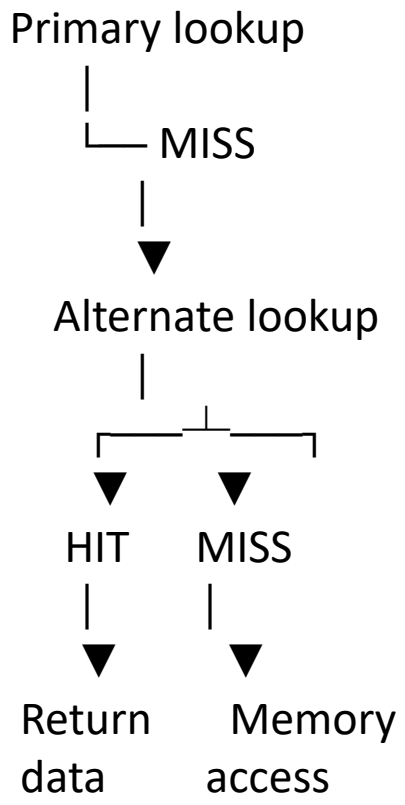
- The cache first performs a normal direct-mapped lookup.

- If it hits, the access is as fast as a direct-mapped cache.

Step 2 — Check alternate location

- If the primary lookup misses, the cache calculates an **alternate location** and checks it.

- Therefore, the second location is accessed **sequentially**, rather than simultaneously.



Example

- Suppose we have a pseudo-associative cache:

Cache
_____

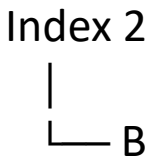
Index 0 → Primary / Alternate
Index 1 → Primary / Alternate
Index 2 → Primary / Alternate
Index 3 → Primary / Alternate

Suppose memory blocks **A** and **B** both map to Index 2.

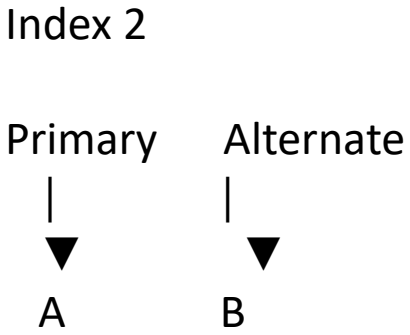
A conventional direct-mapped cache might have:

Index 2
|
└── A

- When B arrives:
- A gets evicted.



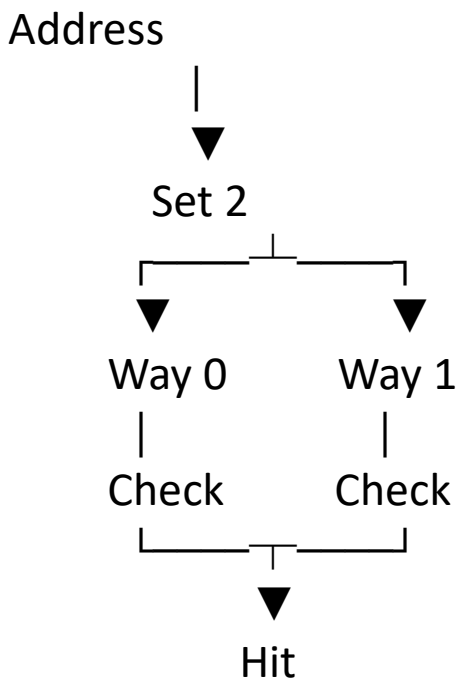
- A pseudo-associative cache can place them in different locations:



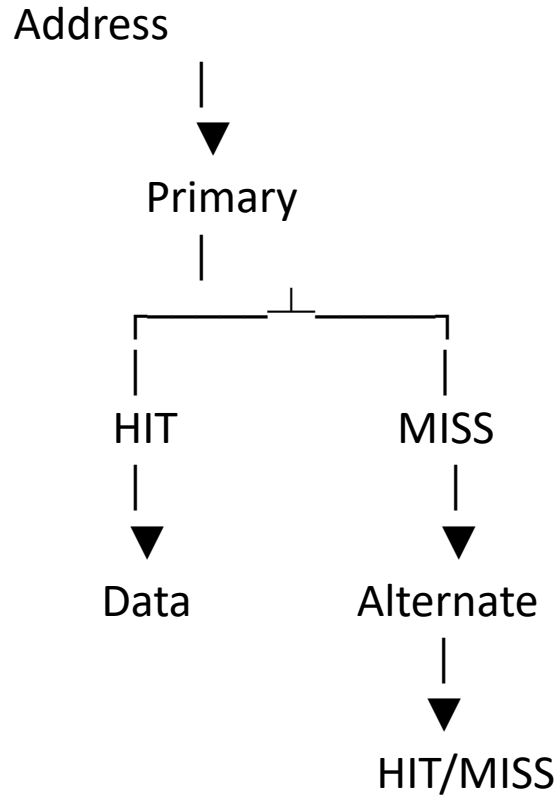
Now both blocks can coexist.

Why "Pseudo"?

- t is called **pseudo-associative** because it provides some associative behavior, but it is not a conventional set-associative cache.
- Compare:
- **2-way set-associative cache**
- Both ways can be checked simultaneously:



- **Pseudo-associative cache**
- The ways are checked sequentially:



This sequential behavior is the key distinction.

Comparison

Feature	Direct-Mapped	Pseudo-Associative	2-Way Set-Associative
Possible locations	1	2	2
Lookup	One location	Primary → alternate	Both ways usually in parallel
Normal hit latency	Very low	Very low	Low
Conflict misses	High	Lower	Lower
Hardware complexity	Low	Moderate	Higher
Energy	Low	Low/moderate	Higher
Miss penalty	Normal	Extra lookup if primary misses	Usually no second lookup
Associativity	1-way	Pseudo 2-way	2-way

Loop Interchange

Assumptions for both examples

- Matrix size = **1024 × 1024**
- Element = int = **4 bytes**
- Cache line = **64 bytes**
- Therefore, one cache line contains **16 integers**
- Row-major storage, as in C/C++
- Assume the cache is not large enough to retain all 1024 rows during column-wise traversal.
- We count **compulsory/spatial cache-line misses** and assume no unusual conflict effects.

Before Loop Interchange

```
for (int j = 0; j < 1024; j++) {
   for (int i = 0; i < 1024; i++) {
      sum += A[i][j];
   }
}
```

- The access pattern is **column-wise**:

A[0][j]
A[1][j]
A[2][j]
...
A[1023][j]

Each consecutive access is:

1024 × 4 = 4096 bytes apart

Therefore, almost every access requires a different cache line.

Number of accesses

1024 × 1024 = 1,048,576 accesses

Approximately:

Cache misses ≈ 1,048,576

After Loop Interchange

```
for (int i = 0; i < 1024; i++) {
    for (int j = 0; j < 1024; j++) {
        sum += A[i][j];
    }
}
```

- Now the access is **row-wise:**

A[i][0]
A[i][1]
A[i][2]
...
A[i][1023]

One cache line contains:

64 / 4 = 16 integers

Total number of cache lines needed:

1,048,576 / 16
= 65,536 cache lines

Therefore:

Cache misses ≈ 65,536

Improvement

	Before interchange	After interchange
Memory accesses	1,048,576	1,048,576
Cache-line misses	**≈1,048,576**	**≈65,536**
Cache hits	≈0	≈983,040
Miss reduction	—	**≈93.75%**

So loop interchange reduces the misses from approximately **1.05 million to 65,536**.

Matrix Addition

```
for (int j = 0; j < 1024; j++) {
    for (int i = 0; i < 1024; i++) {
        C[i][j] = A[i][j] + B[i][j];
    }
}
```

There are three matrices:

A → Read
B → Read
C → Write

Before Loop Interchange
The access is column-wise:

A[0][j], A[1][j], A[2][j] ...
B[0][j], B[1][j], B[2][j] ...
C[0][j], C[1][j], C[2][j] ...

Approximately every access to each matrix causes a cache-line miss.
For each matrix:

1,048,576 accesses
≈ 1,048,576 misses

For three matrices:

A misses ≈ 1,048,576
B misses ≈ 1,048,576
C misses ≈ 1,048,576

Total ≈ 3,145,728 misses

After Loop Interchange

```
for (int i = 0; i < 1024; i++) {
    for (int j = 0; j < 1024; j++) {
        C[i][j] = A[i][j] + B[i][j];
    }
}
```

Now each cache line provides **16 consecutive elements**.
For each matrix:

1,048,576 / 16
= 65,536 cache lines

Therefore:

A misses ≈ 65,536
B misses ≈ 65,536
C misses ≈ 65,536

Total ≈ 196,608 misses

Improvement

	Before interchange	After interchange
A cache misses	≈1,048,576	≈65,536
B cache misses	≈1,048,576	≈65,536
C cache misses	≈1,048,576	≈65,536
Total misses	**≈3,145,728**	**≈196,608**
Miss reduction	—	**≈93.75%**

Loop Fusion

Before Loop Fusion

```
for (int i = 0; i < N; i++) {
    A[i] = B[i] + 1;
}

for (int i = 0; i < N; i++) {
    C[i] = A[i] * 2;
}
```

The first loop produces `A[i]`:

B[i] → A[i]

Then the second loop reads `A[i]`:

A[i] → C[i]

The problem is that **the entire first loop finishes before the second loop starts**.
If `N` is large, the cache may no longer contain the values of `A` when the second loop accesses them.

After Loop Fusion

```
for (int i = 0; i < N; i++) {
    A[i] = B[i] + 1;
    C[i] = A[i] * 2;
}
```

Now:

B[i] → A[i] → C[i]
↑
immediate reuse

As soon as `A[i]` is produced, it is used by the next statement.
Therefore, `A[i]` has excellent **temporal locality**.

Numerical Example

- Assume:
- N = 1,048,576
- int = 4 bytes
- Cache line = 64 bytes
- 16 integers per cache line
- A, B, and C are much larger than the cache.

Before Fusion

First loop:

```
for (i = 0; i < N; i++)
    A[i] = B[i] + 1;
```

Approximately:

B → 65,536 cache-line reads
A → 65,536 cache-line writes

Second loop:

```
for (i = 0; i < N; i++)
    C[i] = A[i] * 2;
```

Because the arrays are much larger than the cache, assume `A` has been evicted:

A → 65,536 cache-line reads
C → 65,536 cache-line writes

So approximately:

Total cache-line transfers ≈ 262,144

The important issue is that **A is accessed twice in separate phases**.

After Fusion

```
for (i = 0; i < N; i++) {
    A[i] = B[i] + 1;
    C[i] = A[i] * 2;
}
```

For each cache line:

Load B cache line

↓

Compute A

↓

Use A immediately

↓

Write C

The second access to `A` can often be satisfied from a register or cache rather than requiring another memory access. Thus, the extra cache-line read of `A` can be eliminated.

Approximately:

B → 65,536

A → 65,536

C → 65,536

Total ≈ 196,608

So the simplified comparison is:

	Before Fusion	After Fusion
B cache-line accesses	65,536	65,536
A cache-line accesses	131,072	65,536
C cache-line accesses	65,536	65,536
Total	**262,144**	**196,608**
Reduction	—	**≈25%**

How Blocking Reduces Cache Misses

How Blocking Reduces Cache Misses

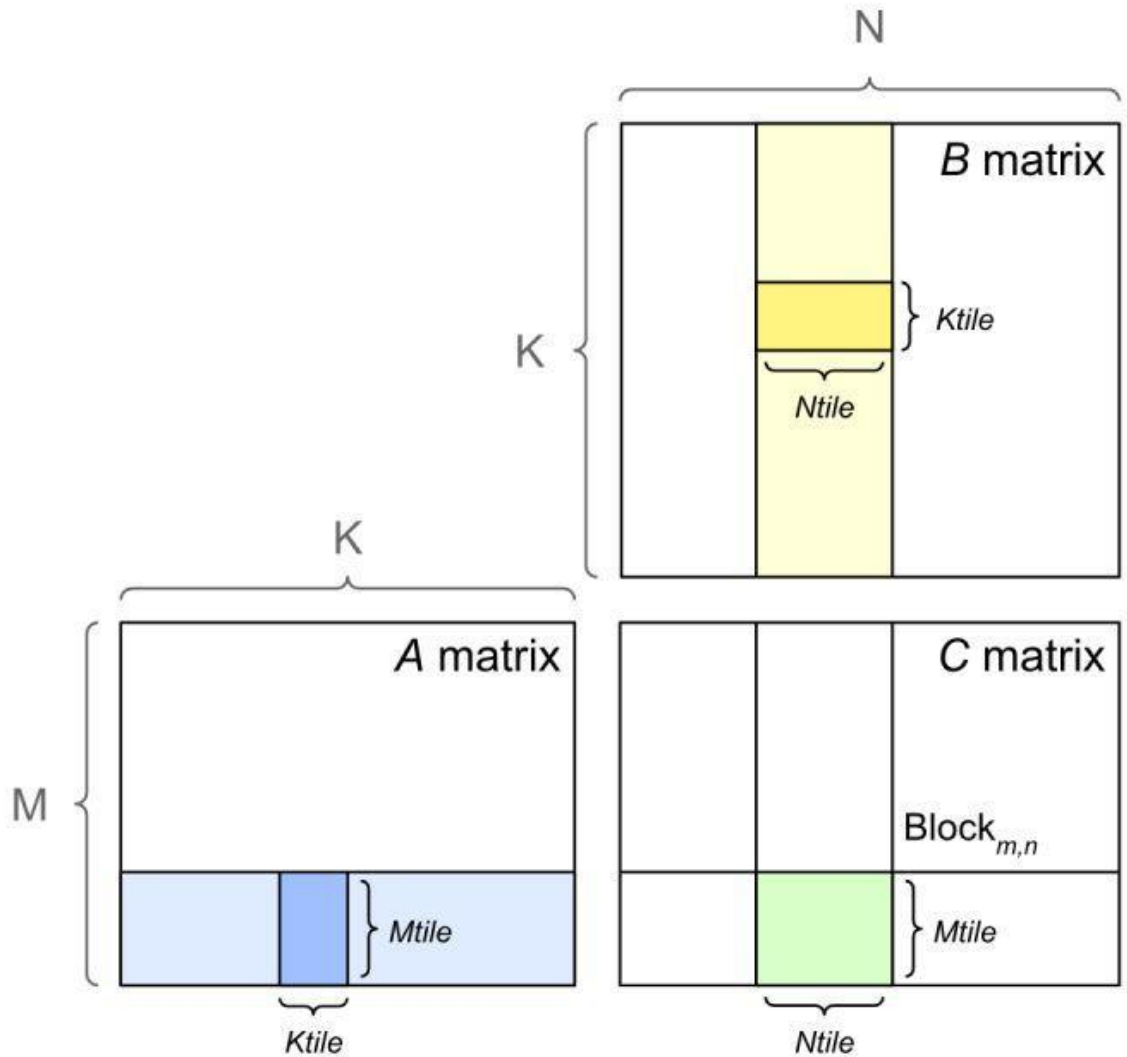
- **Blocking**, also called **tiling**, divides a large computation into smaller blocks (tiles) that fit into the cache.
- The main idea is:
- **Instead of processing the entire large data structure at once, process a small portion that fits in the cache and reuse it before moving to the next portion.**
- This improves **temporal locality** and reduces cache misses.

Matrix Multiplication

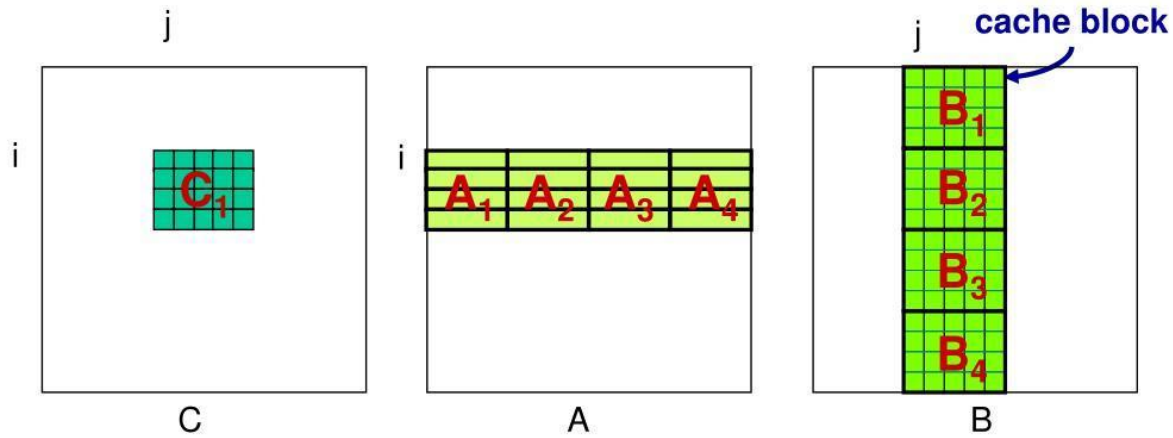
```
for (i = 0; i < N; i++) {
    for (j = 0; j < N; j++) {
        for (k = 0; k < N; k++) {
            C[i][j] += A[i][k] * B[k][j];
        }
    }
}
```

Assume:

- `N = 1024`
- `int = 4 bytes`
- Cache line = **64 bytes**
- 16 integers per cache line
- Matrix size = `1024 × 1024`
- Each matrix = **4 MB**
- Cache = **32 KB**
- Row-major storage


N
B matrix
K
Ktile
Ntile
K
A matrix
C matrix
M
Block$_{m,n}$
Mtile
Mtile
Ktile
Ntile

“Blocked” Matrix Multiplication



Order of the operations:

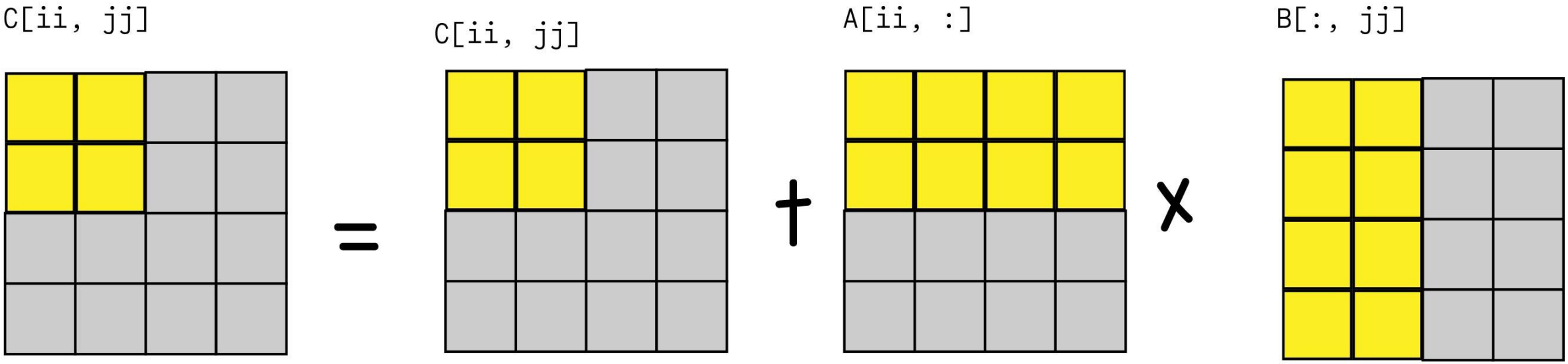
Compute the first r terms of C1 ($=A_1*B_1$)

Compute the next r terms of C1 ($=A_2*B_2$)

. . .

Compute the last r terms of C1 ($=A_4*B_4$)

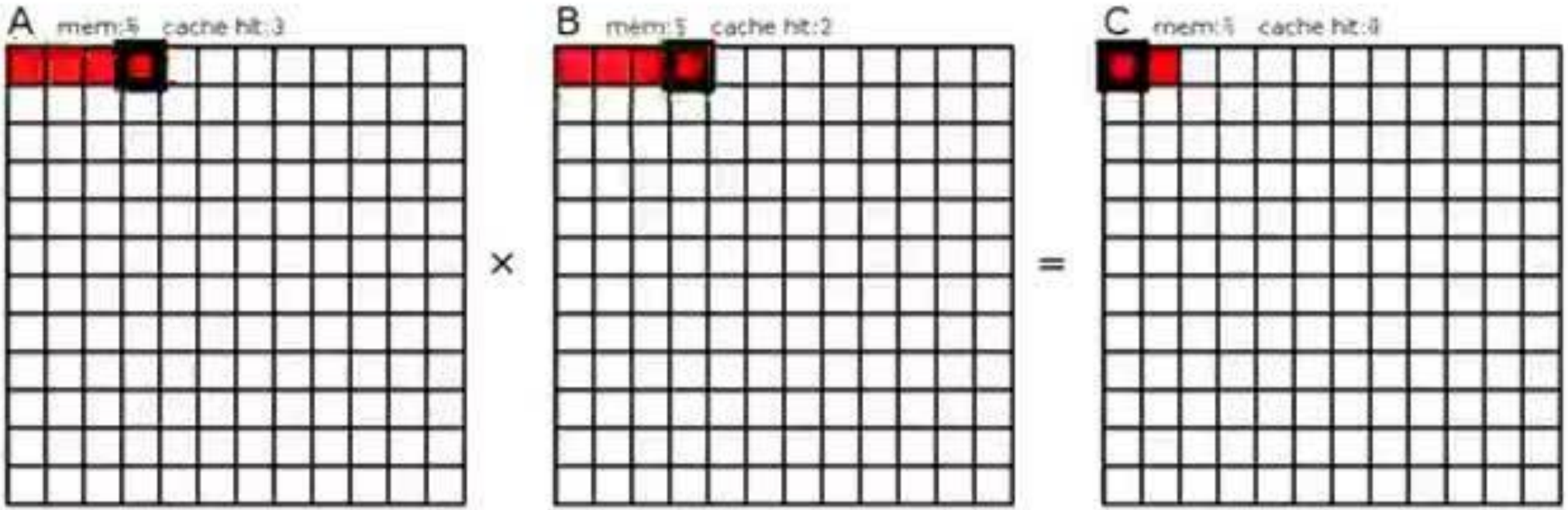
b = 2
(block size)



ii, jj, kk denote block indices while i, j, k denote element indices

```
// C = C + A * B
// b = n / N (where b is the block size)
for ii = 1 to N:
  for jj = 1 to N:
    for kk = 1 to N:
      C[ii, jj] += A[ii, kk] * B[kk, jj]
```

Matrix multiplication: Tiled, B transposed



Totals: mem:12 cache hits:7 ≅58%



Without Blocking

- The important access patterns are:

  A[i][k] → sequential
  B[k][j] → column-wise
  C[i][j] → repeatedly accessed

The problem is particularly severe for `B`.
For example, for a fixed `j`:

B[0][j]
B[1][j]
B[2][j]
B[3][j]
...

Because the matrix is row-major, consecutive elements are **1024 × 4 = 4096 bytes apart**.
Therefore, accessing a column of `B` has very poor spatial locality.

Approximate cache misses

- or the simple i-j-k implementation, the B accesses can result in roughly one cache-line miss per element under a simple cache model:
- $1024^3 = 1{,}073{,}741{,}824$
- Therefore:
- **Approximately 1.07 billion cache misses** can occur for the problematic B accesses.
- This is an illustrative worst-case/simple-model estimate; actual hardware misses depend on cache organization and replacement behavior.

With Blocking

- Now divide the matrices into smaller blocks.
- Suppose we choose:
- $B = 32$
- So instead of processing the entire matrix, we process **32 × 32 tiles**.
- The blocked algorithm becomes:

```
for (int ii = 0; ii < N; ii += 32) {
   for (int jj = 0; jj < N; jj += 32) {
      for (int kk = 0; kk < N; kk += 32) {

         for (int i = ii; i < ii + 32; i++) {
            for (int j = jj; j < jj + 32; j++) {
               for (int k = kk; k < kk + 32; k++) {

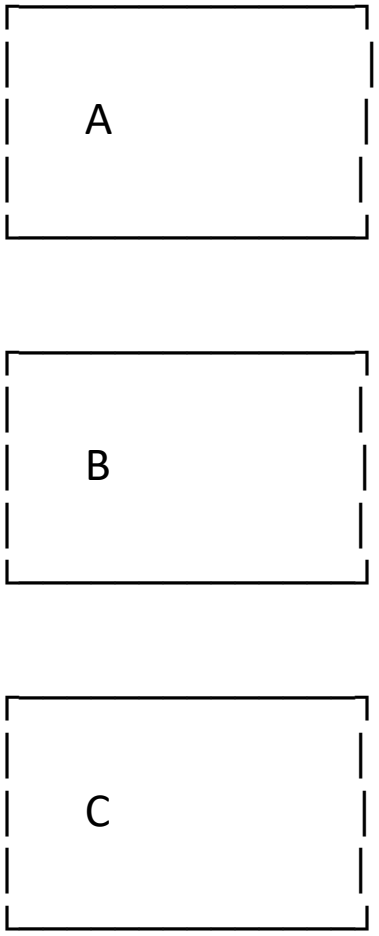
                  C[i][j] += A[i][k] * B[k][j];

               }
            }
         }

      }
   }
}
```

Now the processor works on:

32 × 32 block



Each block contains:

$$32 \times 32 = 1024\ integers$$

or:

$$1024 \times 4 = 4096\ bytes = 4\ KB$$

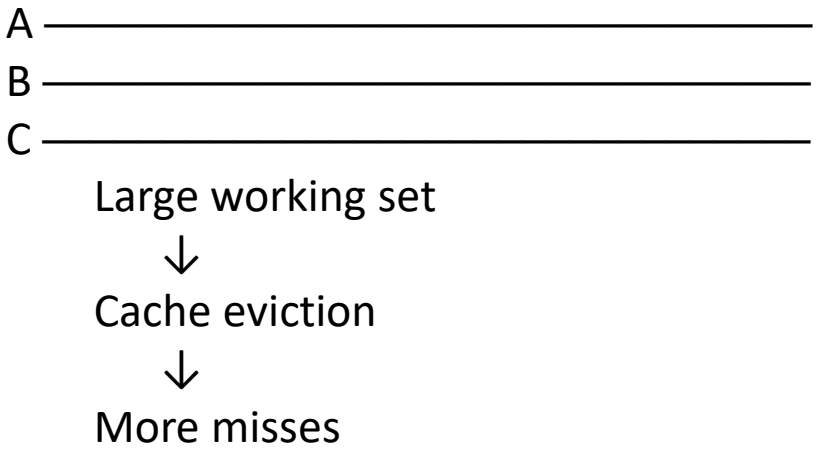
Three blocks require approximately:

$$3 \times 4\ KB = 12\ KB$$

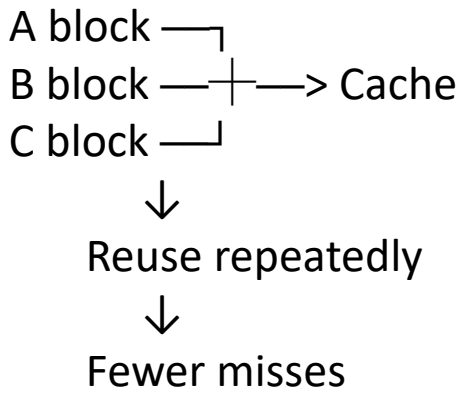
which fits comfortably in a **32 KB cache**.

Why Does Blocking Reduce Misses?

- Without blocking:



With blocking:



The same elements of `A`, `B`, and `C` are reused many times while they are still in the cache.

Numerical Comparison

- Let's use a simplified cache-line model to illustrate the improvement.
- **Matrix size**
- 1024×1024
- **Cache line**
- 64 *bytes* $= 16$ *integers*
- Number of cache lines in one matrix:
- (1024×1024)/16=65,536

- **Without Blocking**
- The B[k][j] column access has a stride of:
- $1024 \times 4 = 4096$ *bytes*
- Thus, under the simplified assumption that each access to B brings in a new cache line:
- $1024^3 = 1{,}073{,}741{,}824$
- approximately:
- **1.07 billion cache-line misses**

- **With 32 × 32 Blocking**
- A 32 × 32 tile contains:
- $32 \times 32 = 1024$
- elements, requiring:
- $1024/16 = 64$
- cache lines per tile.
- There are:
- $1024/32 = 32$
- tiles along each dimension.
- Blocking allows the same tiles to be reused many times.
- A simplified estimate for loading the matrix tiles is:
- $32^3 \times 64$
- $= 2{,}097{,}152$
- cache-line loads/misses for the tiled data movement, ignoring additional effects such as write allocation and edge conditions.

So the illustrative comparison is:

Matrix Multiplication	Cache Misses
Without blocking	**≈ 1,073,741,824**
With 32×32 blocking	**≈ 2,097,152**
Reduction	**≈ 99.8%**